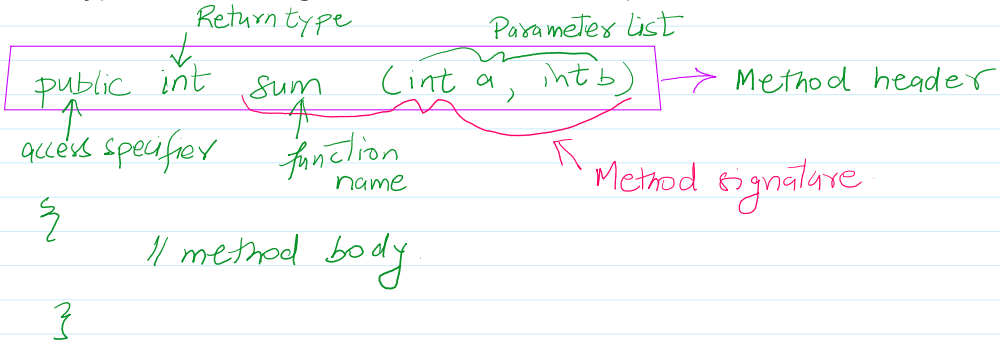
Method in Java

29 August 2024 20:08

In general, a method is a way to perform some task. Similarly, the method in Java is a collection of instructions that performs a specific task. It provides the reusability of code. We can also easily modify code using methods.

What is a method in Java?
A method is a block of code or collection of statements, or a set of code grouped together to perform a certain task or operation. It is used to achieve the reusability of code. We write a method once and use it many times. We do not require to write code again and again. It also provides the easy modification and readability of code, just by adding or removing a chunk of code. The method is executed only when we call or invoke it. The most important method in Java is the main() method.

Method declaration
The method declaration provides information about method attributes such as visibility, return type name and arguments. It has six components that are known as method header:



Method signature: Every method has a method signature. It is a. part of method declaration. It includes the **method name and the parameter list.**

Access specifier: Access specifier or modifier is the access type of the method It specifies the visibility of the method. Java provides four types of access specifier:

- **'public':** The method is accessible by all classes. When we use public specifier in our function declaration.
- **'private':** When we use private access specifier, the method is accessible only in the class in which it is defined.
- **'protected':** When we use protected access specifier, the method is accessible within the same package or subclasses in a different package.
- **'default':** When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.

Return type: Return type is the data type that the method returns. It may have a primitive data type, object, collection, void etc. if the method does not return anything we use void keyword.

Method name: It is a unique name that is used to define the name of a method. It must be corresponding to the functionality of the method. So if we are creating a method for subtraction of two numbers, the method name must be **subtraction()**. A method is invoked by its name.

Parameter list: It is a list of parameters separated by comma and enclosed in a pair of parenthesis. It contains the data type and variable name. If the method has no parameter, left the parenthesis blank.

Method body: It is a part of the method declaration. It contains all the actions to be performed. It is. enclosed within the pair of curly braces.

Method body: It is a part of the method declaration. It contains all the actions to be performed. It is. enclosed within the pair of curly braces.

Types of method
1. Predefined Method
2. User-defined method

return: It the execution control back to the caller line of the function, sometimes with value, or sometimes without value.

x = 93
d = 3

$9\sqrt{93}$

```
public boolean isPrime(int x){
    if(x == 1 || (x>2 && x % 2 == 0)){
        return false;
    }
    for(int d = 3; d <= Math.sqrt(x); d += 2){
        if(x % d == 0){
            return false;
        }
    }
    return true;
}
```

"return" can be written in 3 ways.

- only return;
- return <value/variable>;
- return <condition>;

This return statement is applicable only with functions having void return types

This return statement is only applicable for functions, which is having boolean as return type.

```
public boolean isPrime(int x){
    if(x == 1 || (x>2 && x % 2 == 0)){
        return false;
    }
    for(int d = 3; d <= Math.sqrt(x); d += 2){
        if(x % d == 0){
            return false;
        }
    }
    return true;
}
```

```
package Chapter5Functions;
import java.util.*;

/**
 * Write a description of class TypeOfNumbers here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TypeOfNumbers
{
    public boolean isPrime(int x){
        int count = 1;
        if(x == 1 || (x>2 && x % 2 == 0)){
            System.out.print("\nNumber of steps: " + count);
            return false;
        }

        for(int d = 3; d <= Math.sqrt(x); d += 2, count++){
            if(x % d == 0){
                System.out.print("\nNumber of steps: " + count);
```

count = ~~1~~ ~~2~~ 3

d = ~~3~~ ~~5~~ 7

```
                System.out.print("\nNumber of steps: " + count);
                return false;
            }
        }
        System.out.print("\nNumber of steps: " + count);
        return true;
    }

    public boolean isEvenOdd(int x){
        //if the number is even this method will return true
        //otherwise false
        return (x % 2 == 0);
    }

    public static void main(String args[]){
        //Object creation
        TypeOfNumbers ob = new TypeOfNumbers();
        int N;
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter any number: ");
        N = sc.nextInt();

        if (ob.isPrime(N)){ //function call or invocatio of method
            System.out.print("\nPrime number.");
        }
        else if(ob.isEvenOdd(N)){//function call or invocatio of method
            System.out.print("\nEven number.");
        }
        else{
            System.out.print("\nOdd number.");
        }
    }
}
```